

Calculate Semester Grade

Five Individual Arrays

```
char id[ 4 ][ 8 ];  
char name[ 4 ][ 8 ];  
int homework[ 4 ];  
int midterm[ 4 ];  
int final[ 4 ];
```

id[0]	932201
id[1]	932202
id[2]	932203
id[3]	932204

name[0]	Chen
name[1]	Lee
name[2]	Lin
name[3]	Wu

homework[0]	78
homework[1]	65
homework[2]	90
homework[3]	45

midterm[0]	86
midterm[1]	91
midterm[2]	77
midterm[3]	84

final[0]	93
final[1]	82
final[2]	68
final[3]	58

Array of struct

```
struct Grade
{
    char id[ 8 ];
    char name[ 8 ];
    int midTerm;
    int final;
    int homework;
};

Grade grades[ 4 ];
```

grades[0]

932201	Chen	78	86	93
--------	------	----	----	----

grades[1]

932202	Lee	65	91	82
--------	-----	----	----	----

grades[2]

932203	Lin	90	77	68
--------	-----	----	----	----

grades[3]

932204	Wu	45	84	58
--------	----	----	----	----

```
int main()
{
    const int numStudents = 2;
    char id[ numStudents ][ 8 ];
    char name[ numStudents ][ 8 ];
    int homework[ numStudents ];
    int midTerm[ numStudents ];
    int final[ numStudents ];
    for( int i = 0; i < numStudents; i++ )
    {
        cout << "ID: ";
        cin >> id[i];
        cout << "name: ";
        cin >> name[i];
        cout << "homework grade: ";
        cin >> homework[i];
        cout << "midterm exam grade: ";
        cin >> midTerm[i];
        cout << "final exam grade: ";
        cin >> final[i];
        cout << endl;
    }
    for( int i = 0; i < numStudents; i++ )
        cout << "The semester grade of " << name[i] << " is "
            << ( midTerm[i] + homework[i] ) * 0.2 + final[i] * 0.6
            << endl;
}
```

```
int main()
{
    int i;
    const int numStudents = 2;
    Grade grades[ numStudents ];

    for( i = 0; i < numStudents; i++ )
    {
        cout << "ID: ";
        cin >> grades[i].id;
        cout << "name: ";
        cin >> grades[i].name;
        cout << "midterm exam grade: ";
        cin >> grades[i].midTerm;
        cout << "final exam grade: ";
        cin >> grades[i].final;
        cout << "homework grade: ";
        cin >> grades[i].homework;
        cout << endl;
    }

    for( i = 0; i < numStudents; i++ )
        cout << "The semester grade of " << grades[i].name << " is "
            << ( grades[i].homework + grades[i].midTerm ) * 0.2 +
                grades[i].final * 0.6 << endl;
}
```

Vectors

```
void add( int u1_x, int u1_y, int u2_x, int u2_y, int &u3_x, int &u3_y );  
double length( int u_x, int u_y );
```

```
int main()  
{  
    int v1_x, v1_y, v2_x, v2_y, v3_x, v3_y;  
    v1_x = rand() % 10;  
    v1_y = rand() % 10;  
    v2_x = rand() % 10;  
    v2_y = rand() % 10;  
  
    add( v1_x, v1_y, v2_x, v2_y, v3_x, v3_y );  
    cout << length( v3_x, v3_y ) << endl;  
}
```

```
void add( int u1_x, int u1_y, int u2_x, int u2_y, int &u3_x, int &u3_y )  
{  
    u3_x = u1_x + u2_x;  
    u3_y = u1_y + u2_y ;  
}
```

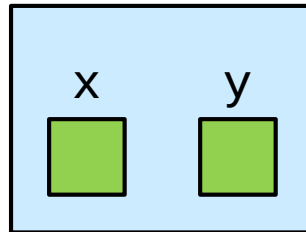
```
double length( int u_x, int u_y )  
{  
    return sqrt( static_cast< double >( u_x * u_x + u_y * u_y ) );  
}
```

```
struct Vector
{
    int x;
    int y;
};
```

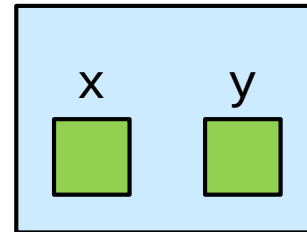
```
void add( Vector u1, Vector u2, Vector &u3 );
double length( Vector u );
```

```
int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    add( v1, v2, v3 );
    cout << length( v3 ) << endl;
}
```

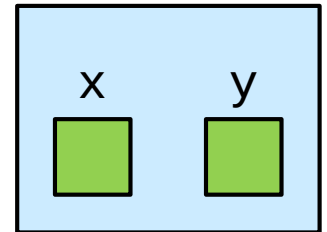
v1



v2



v3



```
void add( Vector u1, Vector u2, Vector &u3 )
{
    u3.x = u1.x + u2.x;
    u3.y = u1.y + u2.y;
}
```



```
struct Vector
{
    int x;
    int y;
};
```

```
Vector add( Vector u1, Vector u2 );
double length( Vector u );
```

```
int main()
{
```

```
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = add( v1, v2 );
    cout << length( v3 ) << endl;
```

```
}
```

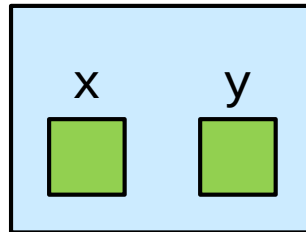
```
Vector add( Vector u1, Vector u2 )
```

```
{
```

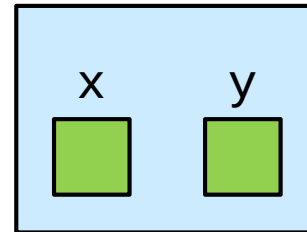
```
    Vector u3;
    u3.x = u1.x + u2.x;
    u3.y = u2.y + u2.y;
    return u3;
```

```
}
```

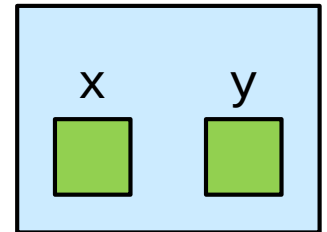
v1



v2



v3



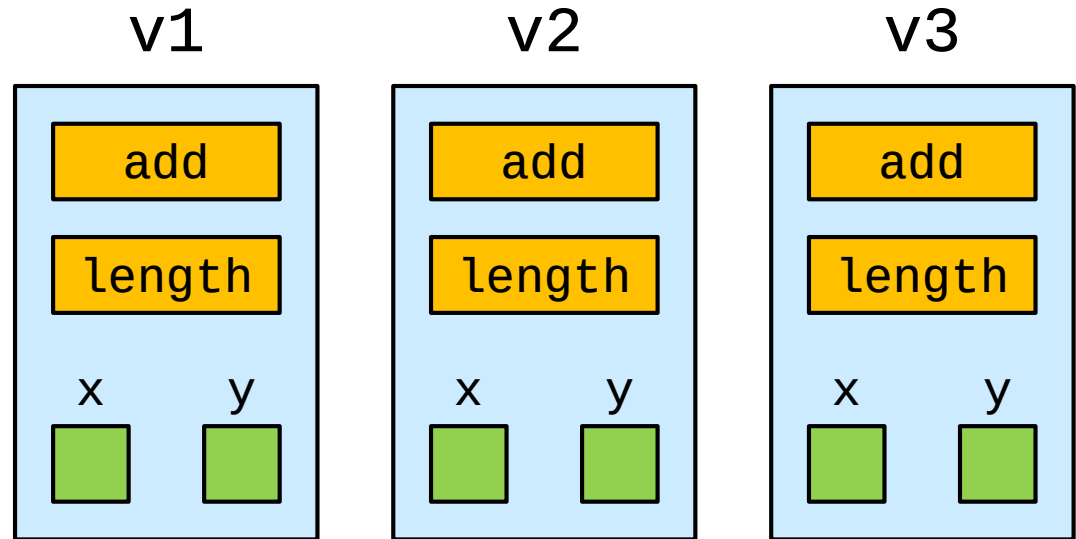
```

class Vector
{
public:
    Vector add( Vector u2 );
    double length();
    int x;
    int y;
};

int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = v1.add( v2 );
    cout << v3.length() << endl;
}

Vector Vector::add( Vector u2 )
{
    Vector u3;
    u3.x = x + u2.x;
    u3.y = y + u2.y;
    return u3;
}

```

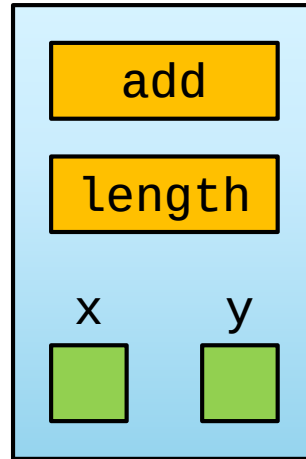


```

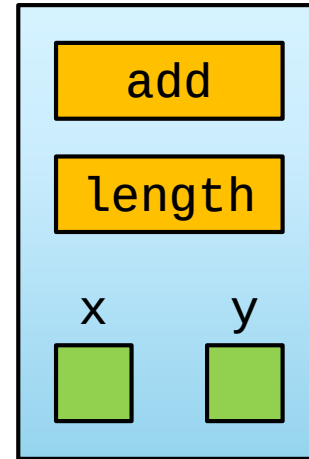
int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = v1.add( v2 );
    cout << v3.length();
}

```

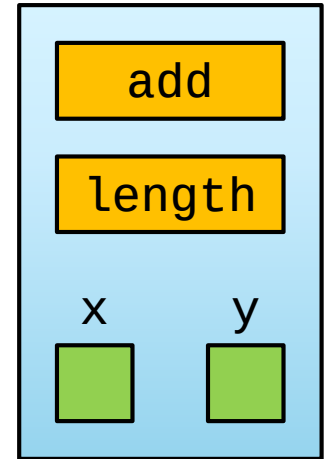
v1



v2



v3



```

double Vector::length()
{
    return sqrt( x*x + y*y );
}

```

```

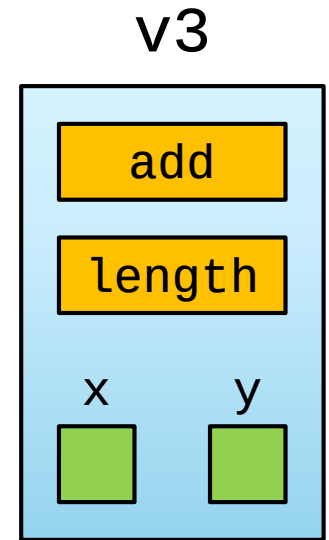
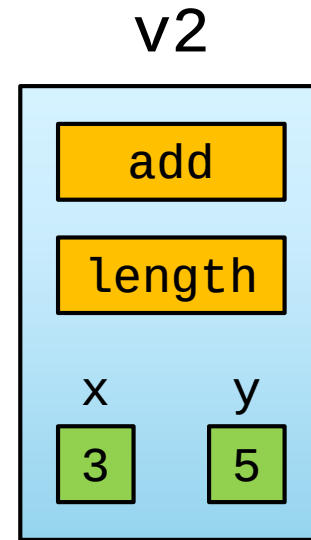
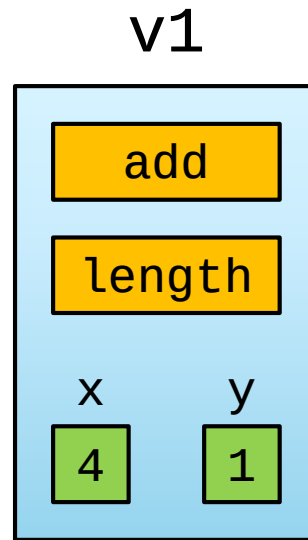
Vector Vector::add( Vector u2 )
{
    Vector u3;
    u3.x = x + u2.x;
    u3.y = y + u2.y;
    return u3;
}

```

```

int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = v1.add( v2 );
    cout << v3.length();
}

```



```

double Vector::length()
{
    return sqrt( x*x + y*y );
}

```

```

Vector Vector::add( Vector u2 )
{
    Vector u3;
    u3.x = x + u2.x;
    u3.y = y + u2.y;
    return u3;
}

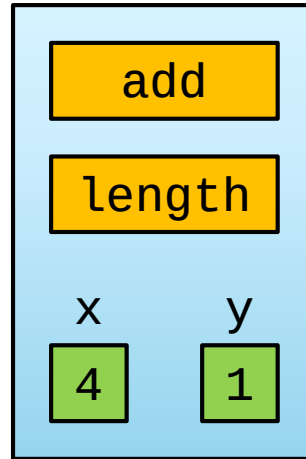
```

```

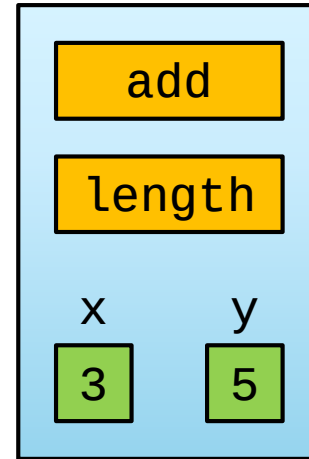
int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = v1.add( v2 );
    cout << v3.length();
}

```

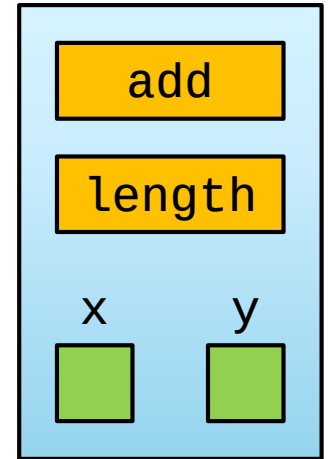
v1



v2



v3



```

double Vector::length()
{
    return sqrt( x*x + y*y );
}

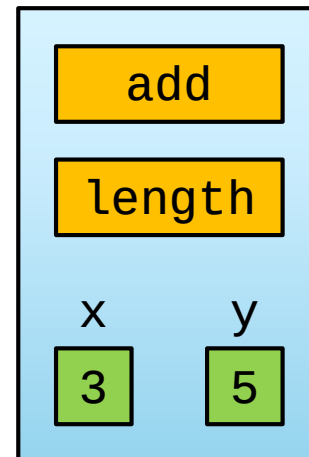
```

```

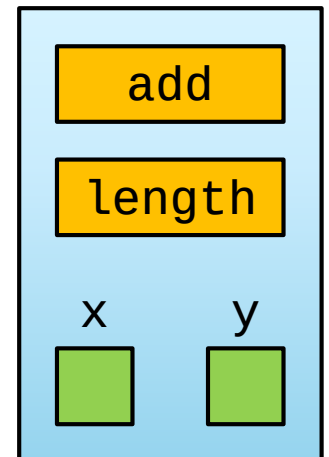
Vector Vector::add( Vector u2 )
{
    Vector u3;
    u3.x = x + u2.x;
    u3.y = y + u2.y;
    return u3;
}

```

u2



u3

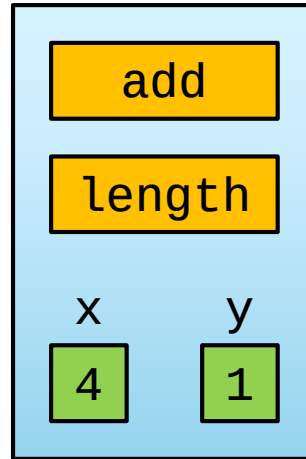


```

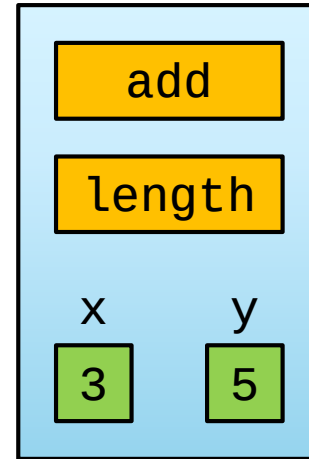
int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = v1.add( v2 );
    cout << v3.length();
}

```

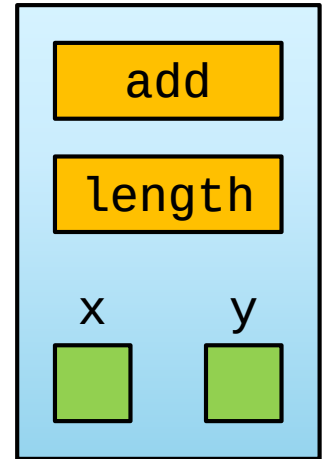
v1



v2



v3



```

double Vector::length()
{
    return sqrt( x*x + y*y );
}

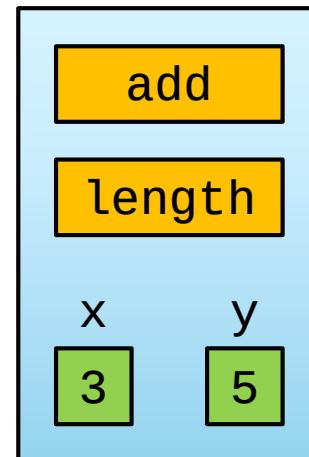
```

```

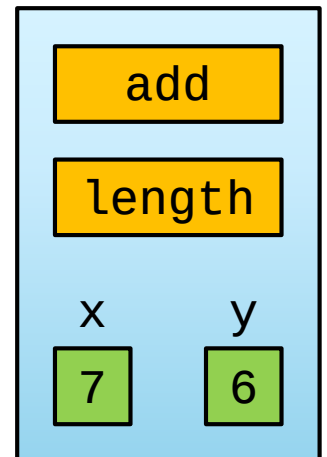
Vector Vector::add( Vector u2 )
{
    Vector u3;
    u3.x = x + u2.x;
    u3.y = y + u2.y;
    return u3;
}

```

u2



u3

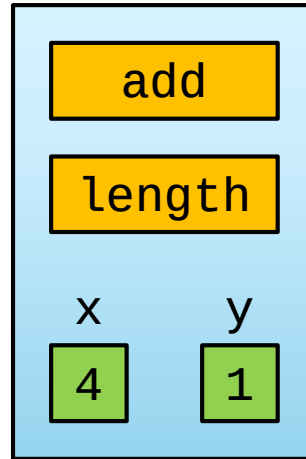


```

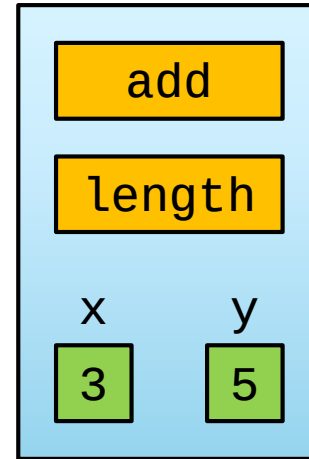
int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = v1.add( v2 );
    cout << v3.length();
}

```

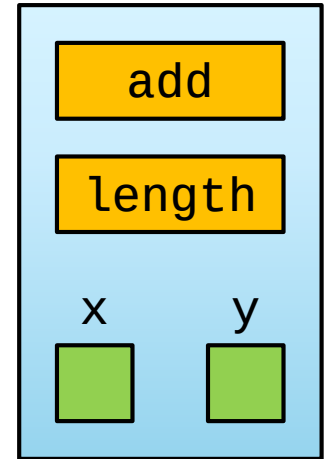
v1



v2



v3



```

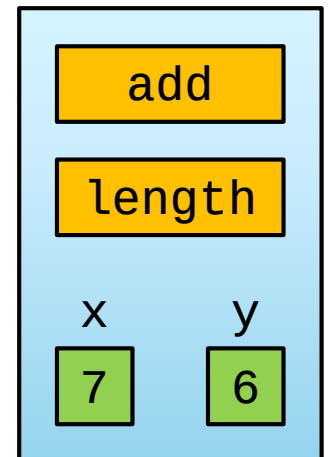
double Vector::length()
{
    return sqrt( x*x + y*y );
}

```

```

Vector Vector::add( Vector u2 )
{
    Vector u3;
    u3.x = x + u2.x;
    u3.y = y + u2.y;
    return u3;
}

```

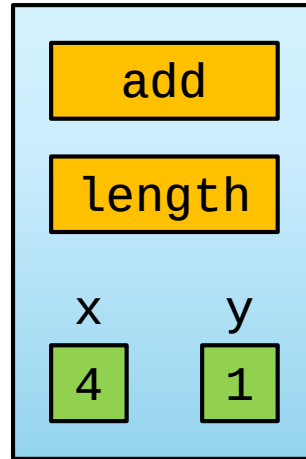


```

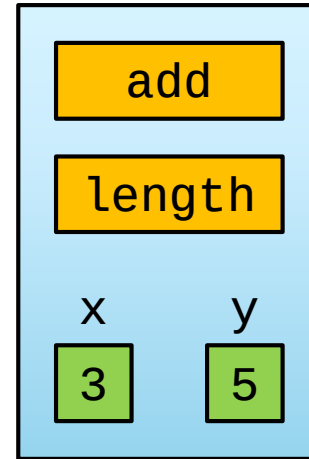
int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = v1.add( v2 );
    cout << v3.length();
}

```

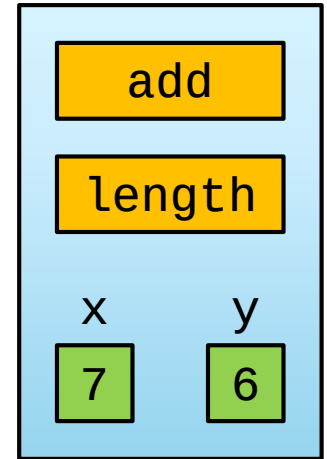
v1



v2



v3



```

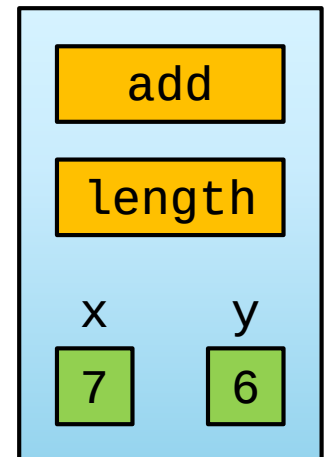
double Vector::length()
{
    return sqrt( x*x + y*y );
}

```

```

Vector Vector::add( Vector u2 )
{
    Vector u3;
    u3.x = x + u2.x;
    u3.y = y + u2.y;
    return u3;
}

```

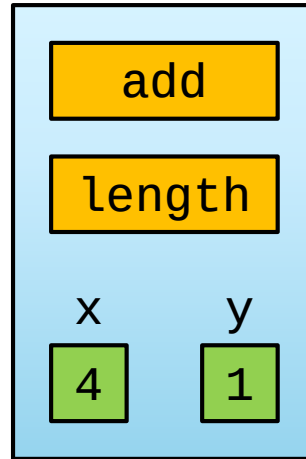



```

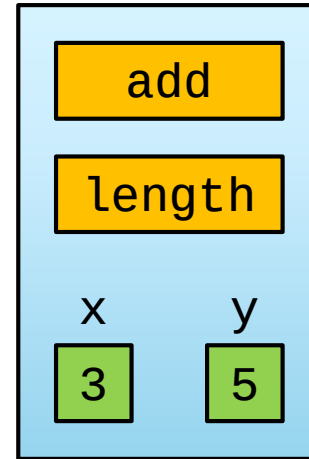
int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = v1.add( v2 );
    cout << v3.length();
}

```

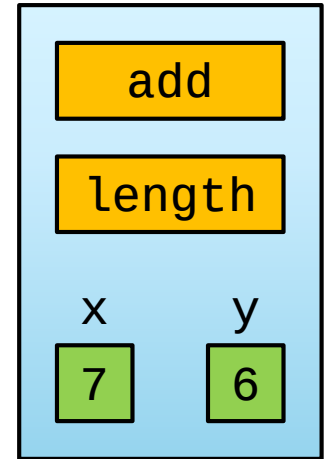
v1



v2



v3



```

double Vector::length()
{
    return sqrt( x*x + y*y );
}

```

```

Vector Vector::add( Vector u2 )
{
    Vector u3;
    u3.x = x + u2.x;
    u3.y = y + u2.y;
    return u3;
}

```

```

struct Vector
{
    int x;
    int y;
};

Vector add( Vector u1, Vector u2 );
double length( Vector u );

int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = add( v1, v2 );
    cout << length( v3 ) << endl;
}

Vector add( Vector u1, Vector u2 )
{
    Vector u3;
    u3.x = u1.x + u2.x;
    u3.y = u2.y + u2.y;
    return u3;
}

```

```

class Vector
{
public:
    Vector add( Vector u2 );
    double length();
    int x;
    int y;
};

int main()
{
    Vector v1, v2, v3;
    v1.x = rand() % 10;
    v1.y = rand() % 10;
    v2.x = rand() % 10;
    v2.y = rand() % 10;
    v3 = v1.add( v2 );
    cout << v3.length() << endl;
}

Vector Vector::add( Vector u2 )
{
    Vector u3;
    u3.x = x + u2.x;
    u3.y = y + u2.y;
    return u3;
}

```

Card shuffling

```
struct card
{
    char *face;
    char *suit;
};

void fillDeck( card wDeck[], char *wFace[], char *wSuit[] );
void shuffle( card wDeck[] );
void deal( card wDeck[] );
```

```
int main()
{
    card deck[ 52 ];

    char *face[ 13 ] =
        { "Ace", "Deuce", "Three", "Four", "Five", "Six", "Seven",
          "Eight", "Nine", "Ten", "Jack", "Queen", "King" };

    char *suit[ 4 ] = { "Hearts", "Diamonds", "Clubs", "Spades" };

    srand( time( 0 ) );

    fillDeck( deck, face, suit );
    shuffle( deck );
    deal( deck );
}

void fillDeck( card wDeck[], char *wFace[], char *wSuit[] )
{
    for( int i = 0; i <= 51; i++ )
    {
        wDeck[ i ].face = wFace[ i % 13 ];
        wDeck[ i ].suit = wSuit[ i / 13 ];
    }
}
```

0	Ace	Hearts
1	Deuce	Hearts
2	Three	Hearts
3	Four	Hearts
4	Five	Hearts
5	Six	Hearts
6	Seven	Hearts
7	Eight	Hearts
8	Nine	Hearts
9	Ten	Hearts
10	Jack	Hearts
11	Queen	Hearts
12	King	Hearts
13	Ace	Diamonds
14	Deuce	Diamonds
15	Three	Diamonds
16	Four	Diamonds
17	Five	Diamonds

18	Six	Diamonds
19	Seven	Diamonds
20	Eight	Diamonds
21	Nine	Diamonds
22	Ten	Diamonds
23	Jack	Diamonds
24	Queen	Diamonds
25	King	Diamonds
26	Ace	Clubs
27	Deuce	Clubs
28	Three	Clubs
29	Four	Clubs
30	Five	Clubs
31	Six	Clubs
32	Seven	Clubs
33	Eight	Clubs
34	Nine	Clubs
35	Ten	Clubs

36	Jack	Clubs
37	Queen	Clubs
38	King	Clubs
39	Ace	Spades
40	Deuce	Spades
41	Three	Spades
42	Four	Spades
43	Five	Spades
44	Six	Spades
45	Seven	Spades
46	Eight	Spades
47	Nine	Spades
48	Ten	Spades
49	Jack	Spades
50	Queen	Spades
51	King	Spades

```
// shuffle cards in deck
void shuffle( card wDeck[] )
```

```
{
    int j;
    card temp;

    for ( int i = 0; i <= 51; i++ )
    {
        j = rand() % 52;
        temp = wDeck[ i ];
        wDeck[ i ] = wDeck[ j ];
        wDeck[ j ] = temp;
    }
}
```

```
// deal cards in deck
```

```
void deal( card wDeck[] )
{
    for ( int i = 0; i <= 51; i++ )
        cout << setw( 5 ) << right << wDeck[ i ].face << " of "
            << setw( 8 ) << left << wDeck[ i ].suit
            << ( i % 2 == 0 ? '\t' : '\n' );
}
```

0	Ace	Hearts
1	Deuce	Hearts
2	Three	Hearts
3	Four	Hearts
4	Five	Hearts
5	Six	Hearts
6	Seven	Hearts
7	Eight	Hearts
8	Nine	Hearts
9	Ten	Hearts
10	Jack	Hearts
11	Queen	Hearts
12	King	Hearts
13	Ace	Diamonds
14	Deuce	Diamonds
15	Three	Diamonds
16	Four	Diamonds
17	Five	Diamonds

18	Six	Diamonds
19	Seven	Diamonds
20	Eight	Diamonds
21	Nine	Diamonds
22	Ten	Diamonds
23	Jack	Diamonds
24	Queen	Diamonds
25	King	Diamonds
26	Ace	Clubs
27	Deuce	Clubs
28	Three	Clubs
29	Four	Clubs
30	Five	Clubs
31	Six	Clubs
32	Seven	Clubs
33	Eight	Clubs
34	Nine	Clubs
35	Ten	Clubs

36	Jack	Clubs
37	Queen	Clubs
38	King	Clubs
39	Ace	Spades
40	Deuce	Spades
41	Three	Spades
42	Four	Spades
43	Five	Spades
44	Six	Spades
45	Seven	Spades
46	Eight	Spades
47	Nine	Spades
48	Ten	Spades
49	Jack	Spades
50	Queen	Spades
51	King	Spades

i j

0	Ace	Hearts
1	Deuce	Hearts
2	Three	Hearts
3	Four	Hearts
4	Five	Hearts
5	Six	Hearts
6	Seven	Hearts
7	Eight	Hearts
8	Nine	Hearts
9	Ten	Hearts
10	Jack	Hearts
11	Queen	Hearts
12	King	Hearts
13	Ace	Diamonds
14	Deuce	Diamonds
15	Three	Diamonds
16	Four	Diamonds
17	Five	Diamonds

18	Six	Diamonds
19	Seven	Diamonds
20	Eight	Diamonds
21	Nine	Diamonds
22	Ten	Diamonds
23	Jack	Diamonds
24	Queen	Diamonds
25	King	Diamonds
26	Ace	Clubs
27	Deuce	Clubs
28	Three	Clubs
29	Four	Clubs
30	Five	Clubs
31	Six	Clubs
32	Seven	Clubs
33	Eight	Clubs
34	Nine	Clubs
35	Ten	Clubs

36	Jack	Clubs
37	Queen	Clubs
38	King	Clubs
39	Ace	Spades
40	Deuce	Spades
41	Three	Spades
42	Four	Spades
43	Five	Spades
44	Six	Spades
45	Seven	Spades
46	Eight	Spades
47	Nine	Spades
48	Ten	Spades
49	Jack	Spades
50	Queen	Spades
51	King	Spades

0	Three	Spades
1	Deuce	Hearts
2	Three	Hearts
3	Four	Hearts
4	Five	Hearts
5	Six	Hearts
6	Seven	Hearts
7	Eight	Hearts
8	Nine	Hearts
9	Ten	Hearts
10	Jack	Hearts
11	Queen	Hearts
12	King	Hearts
13	Ace	Diamonds
14	Deuce	Diamonds
15	Three	Diamonds
16	Four	Diamonds
17	Five	Diamonds

18	Six	Diamonds
19	Seven	Diamonds
20	Eight	Diamonds
21	Nine	Diamonds
22	Ten	Diamonds
23	Jack	Diamonds
24	Queen	Diamonds
25	King	Diamonds
26	Ace	Clubs
27	Deuce	Clubs
28	Three	Clubs
29	Four	Clubs
30	Five	Clubs
31	Six	Clubs
32	Seven	Clubs
33	Eight	Clubs
34	Nine	Clubs
35	Ten	Clubs

36	Jack	Clubs
37	Queen	Clubs
38	King	Clubs
39	Ace	Spades
40	Deuce	Spades
41	Ace	Hearts
42	Four	Spades
43	Five	Spades
44	Six	Spades
45	Seven	Spades
46	Eight	Spades
47	Nine	Spades
48	Ten	Spades
49	Jack	Spades
50	Queen	Spades
51	King	Spades

0	Three	Spades
1	Deuce	Hearts
2	Three	Hearts
3	Four	Hearts
4	Five	Hearts
5	Six	Hearts
6	Seven	Hearts
7	Eight	Hearts
8	Nine	Hearts
9	Ten	Hearts
10	Jack	Hearts
11	Queen	Hearts
12	King	Hearts
13	Ace	Diamonds
14	Deuce	Diamonds
15	Three	Diamonds
16	Four	Diamonds
17	Five	Diamonds

18	Six	Diamonds
19	Seven	Diamonds
20	Eight	Diamonds
21	Nine	Diamonds
22	Ten	Diamonds
23	Jack	Diamonds
24	Queen	Diamonds
25	King	Diamonds
26	Ace	Clubs
27	Deuce	Clubs
28	Three	Clubs
29	Four	Clubs
30	Five	Clubs
31	Deuce	Hearts
32	Seven	Clubs
33	Eight	Clubs
34	Nine	Clubs
35	Ten	Clubs

36	Jack	Clubs
37	Queen	Clubs
38	King	Clubs
39	Ace	Spades
40	Deuce	Spades
41	Ace	Hearts
42	Four	Spades
43	Five	Spades
44	Six	Spades
45	Seven	Spades
46	Eight	Spades
47	Nine	Spades
48	Ten	Spades
49	Jack	Spades
50	Queen	Spades
51	King	Spades

0	Three	Spades
1	Six	Clubs
2	Three	Hearts
3	Four	Hearts
4	Five	Hearts
5	Six	Hearts
6	Seven	Hearts
7	Eight	Hearts
8	Nine	Hearts
9	Ten	Hearts
10	Jack	Hearts
11	Queen	Hearts
12	King	Hearts
13	Ace	Diamonds
14	Deuce	Diamonds
15	Three	Diamonds
16	Four	Diamonds
17	Five	Diamonds

18	Six	Diamonds
19	Seven	Diamonds
20	Eight	Diamonds
21	Nine	Diamonds
22	Ten	Diamonds
23	Jack	Diamonds
24	Queen	Diamonds
25	King	Diamonds
26	Ace	Clubs
27	Deuce	Clubs
28	Three	Clubs
29	Four	Clubs
30	Five	Clubs
31	Six	Clubs
32	Seven	Clubs
33	Eight	Clubs
34	Nine	Clubs
35	Ten	Clubs

36	Jack	Clubs
37	Queen	Clubs
38	King	Clubs
39	Ace	Spades
40	Deuce	Spades
41	Ace	Hearts
42	Four	Spades
43	Five	Spades
44	Six	Spades
45	Seven	Spades
46	Eight	Spades
47	Nine	Spades
48	Ten	Spades
49	Jack	Spades
50	Queen	Spades
51	King	Spades

0	Three	Spades
1	Six	Clubs
2	Three	Hearts
3	Four	Hearts
4	Five	Hearts
5	Six	Hearts
6	Seven	Hearts
7	Eight	Hearts
8	Nine	Hearts
9	Ten	Hearts
10	Jack	Hearts
11	Queen	Hearts
12	King	Hearts
13	Ace	Diamonds
14	Deuce	Diamonds
15	Three	Diamonds
16	Four	Diamonds
17	Five	Diamonds

18	Six	Diamonds
19	Seven	Diamonds
20	Eight	Diamonds
21	Nine	Diamonds
22	Ten	Diamonds
23	Jack	Diamonds
24	Queen	Diamonds
25	King	Diamonds
26	Ace	Clubs
27	Deuce	Clubs
28	Three	Clubs
29	Four	Clubs
30	Five	Clubs
31	Six	Clubs
32	Seven	Clubs
33	Eight	Clubs
34	Nine	Clubs
35	Ten	Clubs

36	Jack	Clubs
37	Queen	Clubs
38	King	Clubs
39	Ace	Spades
40	Deuce	Spades
41	Ace	Hearts
42	Four	Spades
43	Five	Spades
44	Six	Spades
45	Seven	Spades
46	Eight	Spades
47	Nine	Spades
48	Ten	Spades
49	Jack	Spades
50	Queen	Spades
51	King	Spades

0	Three	Spades
1	Six	Clubs
2	Ten	Spades
3	Four	Hearts
4	Five	Hearts
5	Six	Hearts
6	Seven	Hearts
7	Eight	Hearts
8	Nine	Hearts
9	Ten	Hearts
10	Jack	Hearts
11	Queen	Hearts
12	King	Hearts
13	Ace	Diamonds
14	Deuce	Diamonds
15	Three	Diamonds
16	Four	Diamonds
17	Five	Diamonds

18	Six	Diamonds
19	Seven	Diamonds
20	Eight	Diamonds
21	Nine	Diamonds
22	Ten	Diamonds
23	Jack	Diamonds
24	Queen	Diamonds
25	King	Diamonds
26	Ace	Clubs
27	Deuce	Clubs
28	Three	Clubs
29	Four	Clubs
30	Five	Clubs
31	Six	Clubs
32	Seven	Clubs
33	Eight	Clubs
34	Nine	Clubs
35	Ten	Clubs

36	Jack	Clubs
37	Queen	Clubs
38	King	Clubs
39	Ace	Spades
40	Deuce	Spades
41	Ace	Hearts
42	Four	Spades
43	Five	Spades
44	Six	Spades
45	Seven	Spades
46	Eight	Spades
47	Nine	Spades
48	Three	Hearts
49	Jack	Spades
50	Queen	Spades
51	King	Spades

```
int main()
{
    card deck[ 52 ];

    char *face[ 13 ] =
        { "Ace", "Deuce", "Three", "Four", "Five", "Six", "Seven",
          "Eight", "Nine", "Ten", "Jack", "Queen", "King" };

    char *suit[ 4 ] = { "Hearts", "Diamonds", "Clubs", "Spades" };

    srand( time( 0 ) );

    fillDeck( deck, face, suit );
    shuffle( deck );
    deal( deck );
}

void fillDeck( card wDeck[], char *wFace[], char *wSuit[] )
{
    for( int i = 0; i <= 51; i++ )
    {
        wDeck[ i ].face = wFace[ i % 13 ];
        wDeck[ i ].suit = wSuit[ i / 13 ];
    }
}
```

```
// shuffle cards in deck
void shuffle( card wDeck[] )
```

```
{
    int j;
    card temp;

    for ( int i = 0; i <= 51; i++ )
    {
        j = rand() % 52;
        temp = wDeck[ i ];
        wDeck[ i ] = wDeck[ j ];
        wDeck[ j ] = temp;
    }
}
```

```
// deal cards in deck
```

```
void deal( card wDeck[] )
{
    for ( int i = 0; i <= 51; i++ )
        cout << setw( 5 ) << right << wDeck[ i ].face << " of "
            << setw( 8 ) << left << wDeck[ i ].suit
            << ( i % 2 == 0 ? '\t' : '\n' );
}
```


Shuffling without struct

```

int main()
{
    char *deck[ 52 ][ 2 ];

    char *face[ 13 ] =
        { "Ace", "Deuce", "Three", "Four", "Five", "Six", "Seven",
          "Eight", "Nine", "Ten", "Jack", "Queen", "King" };

    char *suit[ 4 ] = { "Hearts", "Diamonds", "Clubs", "Spades" };

    srand( static_cast< unsigned >( time( 0 ) ) );

    fillDeck( deck, face, suit );
    shuffle( deck );
    deal( deck );
}

void fillDeck( char *wDeck[][ 2 ], char *wFace[], char *wSuit[] )
{
    for( int i = 0; i <= 51; i++ )
    {
        wDeck[ i ][ 0 ] = wFace[ i % 13 ];
        wDeck[ i ][ 1 ] = wSuit[ i / 13 ];
    }
}

```

```

// shuffle cards in deck
void shuffle( char *wDeck[][ 2 ] )
{
    int j;
    char *temp;

    for( int i = 0; i <= 51; i++ )
    {
        j = rand() % 52;
        temp = wDeck[ i ][ 0 ];
        wDeck[ i ][ 0 ] = wDeck[ j ][ 0 ];
        wDeck[ j ][ 0 ] = temp;
        temp = wDeck[i][1];
        wDeck[ i ][ 1 ] = wDeck[ j ][ 1 ];
        wDeck[ j ][ 1 ] = temp;
    }
}

// deal cards in deck
void deal( char *wDeck[][ 2 ] )
{
    for( int i = 0; i <= 51; i++ )
        cout << setw( 5 ) << right << wDeck[ i ][ 0 ] << " of "
            << setw( 8 ) << left << wDeck[ i ][ 1 ]
            << ( i % 2 == 0 ? '\t' : '\n' );
}

```